Algoritmi e Principi dell'Informatica – 13-7-2024

Riduzione 30%: ☐
Ritirato/a: ☐

10816454, ZONTA ENRICO

Algoritmi e strutture dati

Esercizio 3 (8 punti) - fare solo il punto 1 se riduzione del 30% della prova

Si definisce *peso di Hamming* il numero di elementi non nulli di una stringa definita su un alfabeto di cifre. Si consideri l'insieme delle stringhe definite dall'espressione regolare $(0|1)^*.2$ sull'alfabeto di cifre $\Sigma = \{0, 1, 2\}$.

1) Si descriva sinteticamente, ma con sufficiente precisione un algoritmo per macchina di Turing a nastro singolo che calcola il peso di Hamming di una stringa dell'insieme sopracitato, rappresentando il risultato in binario, e se ne valutino complessità temporali e spaziali.

0,5

NEL NASTRO SINGOLO SONO GIÀ INSERITE LE STRINGHE DI INPUT.
LEGGO LA PRIMA STRINGA FINO AL 2, CALCOLO IL PESO DI HAMMING,
DOPO AVER LETTO LA PRIMA STRINGA, TORNO ALL'INIZIO DELLA STRINGA E SOVRASCRIVO LA STRINGA INIZIALE LETTA CON IL RISULTATO CALCOLATO. LA COMPLESSITÀ SPAZIALE È PARI ALLA LUNGHEZZA TOTALE DELLE STRINGHE, QUINDI $S(n) = O(n)$ CON n LUNGHEZZA DELL'INPUT.
LA COMPLESSITÀ TEMPORALE È $T(n) = O(n^3)$ IN QUANTO DEVO LEGGERE LA STRINGA, TORNARE ALL'INIZIO DI ESSA E POI SOVRASCRIVERLA.

2) Si descriva una macchina RAM che calcola il peso di Hamming per una stringa dell'insieme sopracitato e stampa il risultato. Se ne valutino complessità temporali e spaziali con l'opportuno criterio di costo.

Esercizio 4 (8 punti)

1) Si scriva lo pseudocodice della procedura che dato un riferimento alla radice di un albero binario di ricerca T, ne "inverte" l'invariante, ovvero restituisce il riferimento alla radice di un albero binario dove tutte le chiavi del sottoalbero destro di un nodo sono minori di quella del nodo, e tutte quelle del sottoalbero sinistro sono maggiori. Il corpo della procedura può allocare al più una quantità costante di variabili scalari. Se ne valuti la complessità temporale.

```
RICERCA (T, x)
1 IF T = NIL OR T.key = x.key
2     RETURN T
3 IF T.key < x.key
4     RETURN RICERCA(T.right, x)
5 ELSE RETURN RICERCA (T.left, x)

INVERT(T, x)
RICERCA(T, x)
IF T.right.key < x.key
    RETURN T.key
IF T.left.key > x.key
    RETURN T.key
```

COMPLESSITÀ TEMPORALE:
$T(n) = O(n)$ CON n IL NUMERO DI NODI

1

2) Si consideri la famiglia di grafi non orientati $\{G_1, \ldots, G_n\}$ con le seguenti caratteristiche. Ogni nodo di un grafo G_k, con $1 \leq k \leq n$, è etichettato con una delle combinazioni semplici (ovvero senza ripetizioni) di k elementi scelti tra $\{1, \ldots, n\}$ e, per ogni combinazione semplice α di k elementi, c'è un nodo in G_k etichettato con α. Due nodi sono adiacenti se le combinazioni con cui sono etichettati differiscono per un solo elemento.

1. Si individui, tra le rappresentazioni viste a lezione, quella più efficiente in memoria per questi grafi.
2. Si riporti la complessità computazionale di determinare se due nodi sono adiacenti.

1. LA RAPPRESENTAZIONE PIÙ EFFICIENTE IN TERMINI DI MEMORIA È LA MACCHINA RAM. POICHÉ LA COMPLESSITÀ COMPUTAZIONALE DELLA RAM È DI ORDINE COSTANTE O LINEARE.

2. PER DETERMINARE SE 2 NODI SONO ADIACENTI È SUFFICIENTE CONFRONTARE L'ETICHETTA ASSOCIATA AI NODI $\alpha = \binom{n}{k} = \frac{n!}{k!(n-k)!}$, IL CONFRONTO DIPENDE DAL NUMERO DI ELEMENTI, PERTANTO LA COMPLESSITÀ COMPUTAZIONALE SARÀ DI ORDINE LINEARE.
$T(n) = O(n)$

1